

Profiling

The profiling module is module ID 8. It manages low-overhead hardware-counter metrics, determines which metrics can be collected together, and coordinates access to profiling resources. The module is loaded when profiling fields or metric-group information are first requested.

Use [dcgmi profile](#) to inspect compatible metric groups or pause profiling watches, and [dcgmi dmon](#) to stream profiling fields.

When profiling metrics are useful

Profiling metrics answer broad utilization questions about a running GPU workload: whether the graphics/compute engines are active, how much SM or tensor-pipe activity is observed, whether device memory is busy, and how much PCIe or NVLink traffic crosses the device. They are suited to continuous cluster telemetry and to comparing phases or replicas of an application at low overhead.

These metrics are interval averages, not a kernel trace. They do not identify which source line, CUDA kernel, or instruction caused a value. Start with DCGM to locate an underused or imbalanced device, then use a developer profiler such as Nsight Systems or Nsight Compute when the investigation needs application-level attribution. Profiling counter resources can conflict, so pause DCGM profiling collection before the developer tool takes ownership and resume it afterward.

Profiling builds on core field watches. `dcgmi profile` discovers which profiling fields the selected GPU supports and how older hardware groups counters; `dcgmi dmon` watches and prints the selected fields. The `dcgmproftester` executable is an optional active load generator for validating the measurement path—it is not required to monitor a real application.

Example: characterize an application phase

First ask the target GPU for its runtime profiling catalogue. Do not assume that a field supported by one GPU generation is available on another:

```
$ dcgmi profile --list --entity-id gpu:0
```

Suppose the output includes field 1002 (SM activity) and field 1005 (DRAM activity). Start a one-second stream in one terminal, then run the application on GPU 0 in another terminal:

```
$ dcgmi dmon --entity-id gpu:0 \  
  --field-id 1002,1005 --delay 1000
```

SM activity measures the fraction of time at least one warp was active on an SM, averaged over all SMs. DRAM activity measures the fraction of cycles in which device-memory traffic was present. A high value in one field does not alone prove that the application is compute- or memory-bound; compare several related metrics over a representative interval and correlate them with application throughput.

If a developer profiler needs the same hardware resources, coordinate with all monitoring clients and pause collection for the entire host engine:

```
$ dcgmi profile --pause
```

Run the developer profiler, then always restore DCGM collection:

```
$ dcgmi profile --resume
```

While paused, profiling watches return blank values. Pause and resume are host-engine-wide, not scoped to GPU 0 or to the client that issued the command.

As GPU-enabled servers become more common in the datacenter, it becomes important to better understand applications’ performance and the utilization of GPU resources in the cluster. Profiling metrics in DCGM enables the collection of a set of metrics using the hardware counters on the GPU. DCGM provides access to device-level metrics at low performance overhead in a continuous manner. This feature is supported in production starting with DCGM 1.7.

The profiling metrics are available as fields through the C, Python, and Go interfaces and through `dcgmi`. The `dcgmproftester` load generator can produce controlled activity for observing and validating those metrics.

Metrics

The following profiling metrics are supported. The definitions and corresponding DCGM field IDs are listed.

Device Level GPU Metrics

Metric	Definition	DCGM Field Name and ID
Graphics Engine Activity	The fraction of time any portion of the graphics or compute engines were active. The graphics engine is active if a graphics/compute context is bound and the graphics/compute pipe is busy. The value	<code>DCGM_FI_PROF_GR_ENGINE_UTIL_RATIO</code> (ID: 1001)

Metric	Definition	DCGM Field Name and ID
	represents an average over a time interval and is not an instantaneous value.	
SM Activity	The fraction of time at least one warp was active on a multiprocessor, averaged over all multiprocessors. Note that “active” does not necessarily mean a warp is actively computing. For instance, warps waiting on memory requests are considered active. The value represents an average over a time interval and is not an instantaneous value. A value of 0.8 or greater is necessary, but not sufficient, for effective use of the GPU. A value less than 0.5 likely indicates ineffective GPU usage. Given a simplified GPU architectural view, if a GPU has N SMs then a kernel using N blocks that runs over the entire time interval will correspond to an activity of 1 (100%). A kernel using N/5 blocks that runs over the entire time interval will correspond to an activity of 0.2 (20%). A kernel using N blocks that runs over one fifth of the time interval, with the SMs otherwise idle, will also have an activity of 0.2 (20%). The value is insensitive to the number of threads per block (see <code>DCGM_FI_PROF_SM_OCCUPANCY_RATIO</code>).	<code>DCGM_FI_PROF_SM_UTIL_RATIO</code> (ID: 1002)
SM Occupancy	The fraction of resident warps on a multiprocessor, relative to the maximum number of concurrent warps supported on a multiprocessor. The value represents an average over a time interval and is not an instantaneous value. Higher occupancy does not necessarily indicate better GPU usage. For GPU memory bandwidth limited workloads (see <code>DCGM_FI_PROF_DRAM_UTIL_RATIO</code>), higher occupancy is indicative of more effective	<code>DCGM_FI_PROF_SM_OCCUPANCY_RATIO</code> (ID: 1003)

Metric	Definition	DCGM Field Name and ID
	GPU usage. However if the workload is compute limited (i.e. not GPU memory bandwidth or latency limited), then higher occupancy does not necessarily correlate with more effective GPU usage. Calculating occupancy is not simple and depends on factors such as the GPU properties, the number of threads per block, registers per thread, and shared memory per block. Use the CUDA Occupancy Calculator to explore various occupancy scenarios.	
Tensor Activity	The fraction of cycles the tensor (HMMA / IMMA) pipe was active. The value represents an average over a time interval and is not an instantaneous value. Higher values indicate higher utilization of the Tensor Cores. An activity of 1 (100%) is equivalent to issuing a tensor instruction every other cycle for the entire time interval. An activity of 0.2 (20%) could indicate 20% of the SMs are at 100% utilization over the entire time period, 100% of the SMs are at 20% utilization over the entire time period, 100% of the SMs are at 100% utilization for 20% of the time period, or any combination in between (see DCGM_FI_PROF_SM_UTIL_RATIO to help disambiguate these possibilities).	DCGM_FI_PROF_TENSOR_UTIL_RATIO (ID: 1004)
FP64 Engine Activity	The fraction of cycles the FP64 (double precision) pipe was active. The value represents an average over a time interval and is not an instantaneous value. Higher values indicate higher utilization of the FP64 cores. An activity of 1 (100%) is equivalent to a FP64 instruction on every SM every fourth cycle on Volta over the entire time interval. An activity of 0.2 (20%) could indicate 20% of the SMs are at 100%	DCGM_FI_PROF_FP64_UTIL_RATIO (ID: 1006)

Metric	Definition	DCGM Field Name and ID
	utilization over the entire time period, 100% of the SMs are at 20% utilization over the entire time period, 100% of the SMs are at 100% utilization for 20% of the time period, or any combination in between (see <code>DCGM_FI_PROF_SM_UTIL_RATIO</code> to help disambiguate these possibilities).	
FP32 Engine Activity	The fraction of cycles the FMA (FP32 (single precision), and integer) pipe was active. The value represents an average over a time interval and is not an instantaneous value. Higher values indicate higher utilization of the FP32 cores. An activity of 1 (100%) is equivalent to a FP32 instruction every other cycle over the entire time interval. An activity of 0.2 (20%) could indicate 20% of the SMs are at 100% utilization over the entire time period, 100% of the SMs are at 20% utilization over the entire time period, 100% of the SMs are at 100% utilization for 20% of the time period, or any combination in between (see <code>DCGM_FI_PROF_SM_UTIL_RATIO</code> to help disambiguate these possibilities).	<code>DCGM_FI_PROF_FP32_UTIL_RATIO</code> (ID: 1007)
FP16 Engine Activity	The fraction of cycles the FP16 (half precision) pipe was active. The value represents an average over a time interval and is not an instantaneous value. Higher values indicate higher utilization of the FP16 cores. An activity of 1 (100%) is equivalent to a FP16 instruction every other cycle over the entire time interval. An activity of 0.2 (20%) could indicate 20% of the SMs are at 100% utilization over the entire time period, 100% of the SMs are at 20% utilization over the entire time period, 100% of the SMs are at 100% utilization for 20% of the time period, or any combination in between (see	<code>DCGM_FI_PROF_FP16_UTIL_RATIO</code> (ID: 1008)

Metric	Definition	DCGM Field Name and ID
	<code>DCGM_FI_PROF_SM_UTIL_RATIO</code> to help disambiguate these possibilities).	
Memory BW Utilization	The fraction of cycles where data was sent to or received from device memory. The value represents an average over a time interval and is not an instantaneous value. Higher values indicate higher utilization of device memory. An activity of 1 (100%) is equivalent to a DRAM instruction every cycle over the entire time interval (in practice a peak of ~0.8 (80%) is the maximum achievable). An activity of 0.2 (20%) indicates that 20% of the cycles are reading from or writing to device memory over the time interval.	<code>DCGM_FI_PROF_DRAM_UTIL_RATIO</code> (ID: 1005)
NVLink Bandwidth	The rate of data transmitted / received over NVLink, not including protocol headers, in bytes per second. The value represents an average over a time interval and is not an instantaneous value. The rate is averaged over the time interval. For example, if 1 GB of data is transferred over 1 second, the rate is 1 GB/s regardless of the data transferred at a constant rate or in bursts. The theoretical maximum NVLink Gen2 bandwidth is 25 GB/s per link per direction.	<code>DCGM_FI_PROF_NVLINK_TX_BYTES</code> (ID: 1011) and <code>DCGM_FI_PROF_NVLINK_RX_BYTES</code> (ID: 1012)
PCIe Bandwidth	The rate of data transmitted / received over the PCIe bus, including both protocol headers and data payloads, in bytes per second. The value represents an average over a time interval and is not an instantaneous value. The rate is averaged over the time interval. For example, if 1 GB of data is transferred over 1 second, the rate is 1 GB/s regardless of the data transferred at a constant rate or in bursts. The theoretical maximum PCIe Gen3 bandwidth is 985 MB/s per lane.	<code>DCGM_FI_PROF_PCIE_TX_BYTES</code> (ID: 1009) and <code>DCGM_FI_PROF_PCIE_RX_BYTES</code> (ID: 1010)

Per-link NVLink profiling

The NVLink transmit and receive metrics can answer two different questions. Watch a GPU to measure its total traffic, or watch individual links to find an idle, imbalanced, or unexpectedly busy path. Use the same transmit and receive field IDs for both views. See [dcgmi profile](#) and [dcgmi dmon](#) for the exact entity syntax and [Field Identifiers](#) for the programmatic migration from fixed per-link fields.

Profiling of the GPU counters requires administrator privileges starting with Linux drivers 418.43 or later. This is documented [here](#). When using profiling metrics from DCGM, ensure that nv-hostengine is started with superuser privileges.

Multiplexing of Profiling Counters

Some metrics require multiple passes to be collected and therefore all metrics cannot be collected together. Due to hardware limitations on the GPUs, only certain groups of metrics can be read together. For example, SM Activity | SM Occupancy cannot be collected together with Tensor Utilization on V100 but can be done on T4. To overcome these hardware limitations, DCGM supports automatic multiplexing of metrics by statistically sampling the requested metrics and performing the groupings internally. This may be transparent to users who requested metrics that may not have been able to be collected together.

A side-effect of multiplexing is that collection at higher frequencies will result in zeroes returned as DCGM attempts to group metrics together for collection.

The metrics that can be collected together for a specific GPU can be determined by running the following command:

```
$ dcgmi profile -l -i 0
```

Group.Subgroup	Field ID	Field Tag
A.1	1002	sm_active
A.1	1003	sm_occupancy
A.1	1004	tensor_active
A.1	1007	fp32_active
A.2	1006	fp64_active
A.3	1008	fp16_active
B.0	1005	dram_active
C.0	1009	pcie_tx_bytes
C.0	1010	pcie_rx_bytes
D.0	1001	gr_engine_active
E.0	1011	nvlink_tx_bytes
E.0	1012	nvlink_rx_bytes

From the output above, we can determine that for this GPU (in this example, an NVIDIA T4), a metric from each letter group can be collected without multiplexing. From this example, a metric from A.1

can be collected with another metric from `A.1` without multiplexing. A metric from `A.1` will be multiplexed with another metric from `A.2` or `A.3`. Metrics from different letter groups can be combined for concurrent collection (without requiring multiplexing by DCGM).

Building on this example further, on T4 these metrics can be collected together without multiplexing:

```
sm_active + sm_occupancy + tensor_active + fp32_active
```

The above DCGM command will show what groupings are supported by the hardware for concurrent collection.

Concurrent Usage of NVIDIA Profiling Tools

Due to current hardware limitations, collection of profiling metrics with DCGM will conflict with usage of other developer tools from NVIDIA such as Nsight Systems or Nsight Compute. Users may encounter an error from DCGM (either with `dcgmi` or when using the APIs) such as:

```
Error setting watches. Result: The requested operation could not be completed because the
```

To allow DCGM to co-exist with the usage of other profiling tools, it is recommended to pause metrics collection with DCGM when the tools are in use and then resume after the usage of the tools is complete.

With `dcgmi profile`, the `--pause` and `--resume` options can be used:

```
$ dcgmi profile --pause
$ dcgmi profile --resume
```

When using DCGM APIs, the following APIs can be called from the monitoring process:

`dcgmProfPause()` and `dcgmProfResume()`

When paused, DCGM will publish `BLANK` values for profiling metrics. These `BLANK` values can be tested with `DCGM_FP64_IS_BLANK(value)` in the C or Python bindings.

CUDA Test Generator (dcgmproftester)

`dcgmproftester` is a CUDA load generator. It can be used to generate deterministic CUDA workloads for reading and validating GPU metrics. The tool is shipped as a simple x86_64 Linux binary along with the CUDA kernels compiled to PTX. Customers can use the tool in conjunction with `dcgmi` to quickly generate a load on the GPU and view the metrics reported by DCGM via `dcgmi dmon` on stdout.

Current packages provide `dcgmproftester11`, `dcgmproftester12`, and `dcgmproftester13`. The number at the end is the CUDA major version associated with that binary, not the application version. See [dcgmproftester](#) for binary selection, the implemented field catalogue, all options, MIG and NVLink scope, output files, and exit-status caveats.

`dcgmproftester` takes two important arguments as input: `-t` for generating load for a particular metric (for example use 1004 to generate a half-precision matrix-multiply-accumulate for the Tensor Cores) and `-d` for specifying the test duration. Add `--no-dcgm-validation` to let `dcgmproftester` generate test loads only.

For a list of all the field IDs that can be used to generate specific test loads, see the table in the Profiling Metrics section. The rest of this section includes some examples using the `dcgmi` command line utility.

For example, generate a Tensor Core load on GPU 0 of an A100 system for 10 seconds. The sample output reaches close to 253 TFLOPS of FP16 Tensor Core performance.

```
$ /usr/bin/dcgmproftester11 --no-dcgm-validation -i 0 -t 1004 -d 10
```

```
Skipping CreateDcgmGroups() since DCGM validation is disabled
Skipping CreateDcgmGroups() since DCGM validation is disabled
Skipping WatchFields() since DCGM validation is disabled
Skipping CreateDcgmGroups() since DCGM validation is disabled
Worker 0:0[1004]: TensorEngineActive: generated ???, dcgm 0.000 (250362.2 gflops)
Worker 0:0[1004]: TensorEngineActive: generated ???, dcgm 0.000 (252917.0 gflops)
Worker 0:0[1004]: TensorEngineActive: generated ???, dcgm 0.000 (253971.7 gflops)
Worker 0:0[1004]: TensorEngineActive: generated ???, dcgm 0.000 (253700.2 gflops)
Worker 0:0[1004]: TensorEngineActive: generated ???, dcgm 0.000 (252599.0 gflops)
Worker 0:0[1004]: TensorEngineActive: generated ???, dcgm 0.000 (253134.6 gflops)
Worker 0:0[1004]: TensorEngineActive: generated ???, dcgm 0.000 (252676.7 gflops)
Worker 0:0[1004]: TensorEngineActive: generated ???, dcgm 0.000 (252861.4 gflops)
Worker 0:0[1004]: TensorEngineActive: generated ???, dcgm 0.000 (252764.1 gflops)
Worker 0:0[1004]: TensorEngineActive: generated ???, dcgm 0.000 (253109.4 gflops)
Worker 0:0[1004]: Message: Bus ID 00000000:00:04.0 mapped to cuda device ID 0
DCGM CudaContext Init completed successfully.
```

```
CU_DEVICE_ATTRIBUTE_MAX_THREADS_PER_MULTIPROCESSOR: 2048
CUDA_VISIBLE_DEVICES:
CU_DEVICE_ATTRIBUTE_MULTIPROCESSOR_COUNT: 108
CU_DEVICE_ATTRIBUTE_MAX_SHARED_MEMORY_PER_MULTIPROCESSOR: 167936
CU_DEVICE_ATTRIBUTE_COMPUTE_CAPABILITY_MAJOR: 8
CU_DEVICE_ATTRIBUTE_COMPUTE_CAPABILITY_MINOR: 0
CU_DEVICE_ATTRIBUTE_GLOBAL_MEMORY_BUS_WIDTH: 5120
CU_DEVICE_ATTRIBUTE_MEMORY_CLOCK_RATE: 1215
Max Memory bandwidth: 1555200000000 bytes (1555.2 GiB)
CU_DEVICE_ATTRIBUTE_ECC_SUPPORT: true
```

In another console, use the `dcgmi dmon -e` command to view the various performance metrics (streamed to stdout) reported by DCGM as the CUDA workload runs on the GPU. In this example, DCGM reports the GPU activity, TensorCore activity and Memory utilization at a frequency of 1Hz (or 1000ms). As can be seen, the GPU is busy doing work (~99% of Graphics Activity showing that the

SMs are busy), with the TensorCore activity pegged to ~93%. Note that `dcgmi` is currently returning the metrics for GPU ID: 0. On a multi-GPU system, you can specify the GPU ID for which DCGM should return the metrics. By default, the metrics are returned for all the GPUs in the system.

```
$ dcgmi dmon -e 1001,1004,1005
```

#	Entity Id	GRACT	TENSO	DRAMA
	GPU 0	0.000	0.000	0.000
	GPU 0	0.000	0.000	0.000
	GPU 0	0.000	0.000	0.000
	GPU 0	0.552	0.527	0.000
	GPU 0	0.969	0.928	0.000
	GPU 0	0.973	0.931	0.000
	GPU 0	0.971	0.929	0.000
	GPU 0	0.969	0.927	0.000
	GPU 0	0.971	0.929	0.000
	GPU 0	0.971	0.930	0.000
	GPU 0	0.973	0.931	0.000
	GPU 0	0.974	0.931	0.000
	GPU 0	0.971	0.930	0.000
	GPU 0	0.974	0.932	0.000
	GPU 0	0.972	0.930	0.000

Metrics on Multi-Instance GPU

The Multi-Instance GPU (MIG) feature allows supported NVIDIA GPUs to be securely partitioned into up to seven separate GPU Instances for CUDA applications, providing multiple users with separate GPU resources for optimal GPU utilization. This feature is particularly beneficial for workloads that do not fully saturate the GPU’s compute capacity and therefore users may want to run different workloads in parallel to maximize utilization. For more information on MIG, refer to the [MIG User Guide](#).

DCGM can provide metrics for workloads that are running on MIG devices. DCGM offers two views of the metrics for MIG:

- GPU device-level metrics
- MIG device (either GPU Instance or Compute Instance) granularity of the metrics

Example 1

In this example, let’s generate a CUDA workload using `dcgmproftester` and observe metrics using `dcgmi dmon`.

In this example, we follow these steps to demonstrate the collection of metrics for MIG devices:

- Create MIG devices (assumes that the GPU has MIG mode enabled)
- Verify DCGM can list the devices
- Create a group of devices for DCGM to monitor
- Run CUDA workloads on the desired MIG device(s)
- Use *dcgmi dmon* to stream metrics

Step 1: Create MIG Devices

Notice that we have partitioned the GPU into two MIG devices with the 3g.20gb profiles.

This device reconfiguration requires administrative GPU access. See [the privilege note](#) and [the process- credentials note](#) if the command is not authorized for the current account.

```
$ sudo nvidia-smi mig -cgi 9,9 -C
```

...

MIG devices:												
GPU	GI ID	CI ID	MIG Dev	Memory-Usage		SM	Vol	Shared				
				BAR1-Usage			Unc	CE	ENC	DEC	OFA	JPG
							ECC					
0	1	0	0	11MiB / 20096MiB		42	0	3	0	2	0	0
				0MiB / 32767MiB								
0	2	0	1	11MiB / 20096MiB		42	0	3	0	2	0	0
				0MiB / 32767MiB								

Step 2: Verify enumeration by DCGM

We can also observe the devices enumerated by DCGM:

```
$ dcgmi discovery -c
```

Instance Hierarchy	
GPU 0	GPU GPU-5fd15f35-e148-2992-4ecb-9825e534f253 (EntityID: 0)
-> I 0/1	GPU Instance (EntityID: 0)
-> CI 0/1/0	Compute Instance (EntityID: 0)
-> I 0/2	GPU Instance (EntityID: 1)
-> CI 0/2/0	Compute Instance (EntityID: 1)

Step 3: Creation of MIG device groups

Create groups of devices for DCGM to monitor. In this example, we will add the GPU and the two GPU Instances (using entity IDs 0 and 1) to the group:

```
$ dcgmi group -c mig-ex1 -a 0,i:0,i:1
```

Successfully created group "mig-ex1" with a group ID of 8
Specified entity ID is valid but unknown: i:0. ParsedResult: ParsedGpu(i:0)
Specified entity ID is valid but unknown: i:1. ParsedResult: ParsedGpu(i:1)
Add to group operation successful.

Now, we can list the devices added to the group and see that the group contains the GPU (GPU:0), GPU Instances (0 and 1):

```
$ dcgmi group -l
```

```
+-----+-----+
| GROUPS                                     |
| 1 group found.                           |
+=====+=====+
| Groups                                    |
| -> 8                                     |
|   -> Group ID      | 8                |
|   -> Group Name    | mig-ex1         |
|   -> Entities      | GPU 0, GPU_I 0, GPU_I 1 |
+-----+-----+
```

Step 4: Run CUDA workloads

Launch `dcgmproftester`. Note that `dcgmproftester` will run on all GPU instances available on the GPU. There is currently no way to limit the GPU instances from `dcgmproftester` (though one could do this by running a `dcgmproftester` container, which we will see in the next example).

```
$ sudo dcmproftester11 --no-dcgm-validation -i 0 -t 1004 -d 120
```

```
Skipping CreateDcgmGroups() since DCGM validation is disabled
Skipping CreateDcgmGroups() since DCGM validation is disabled
Skipping CreateDcgmGroups() since DCGM validation is disabled
Skipping WatchFields() since DCGM validation is disabled
Skipping CreateDcgmGroups() since DCGM validation is disabled
Worker 0:0[1004]: TensorEngineActive: generated ???, dcgm { GPU: 0.000, GI: 0.000, CI: 0.
Worker 0:1[1004]: TensorEngineActive: generated ???, dcgm { GPU: 0.000, GI: 0.000, CI: 0.
Worker 0:0[1004]: TensorEngineActive: generated ???, dcgm { GPU: 0.000, GI: 0.000, CI: 0.
Worker 0:1[1004]: TensorEngineActive: generated ???, dcgm { GPU: 0.000, GI: 0.000, CI: 0.
Worker 0:0[1004]: TensorEngineActive: generated ???, dcgm { GPU: 0.000, GI: 0.000, CI: 0.
Worker 0:1[1004]: TensorEngineActive: generated ???, dcgm { GPU: 0.000, GI: 0.000, CI: 0.
Worker 0:0[1004]: TensorEngineActive: generated ???, dcgm { GPU: 0.000, GI: 0.000, CI: 0.
```

snip...snip

Worker 0:0[1004]: Message: DCGM CudaContext Init completed successfully.

```
CU_DEVICE_ATTRIBUTE_MAX_THREADS_PER_MULTIPROCESSOR: 2048
CUDA_VISIBLE_DEVICES: MIG-GPU-5fd15f35-e148-2992-4ecb-9825e534f253/1/0
CU_DEVICE_ATTRIBUTE_MULTIPROCESSOR_COUNT: 42
CU_DEVICE_ATTRIBUTE_MAX_SHARED_MEMORY_PER_MULTIPROCESSOR: 167936
CU_DEVICE_ATTRIBUTE_COMPUTE_CAPABILITY_MAJOR: 8
CU_DEVICE_ATTRIBUTE_COMPUTE_CAPABILITY_MINOR: 0
CU_DEVICE_ATTRIBUTE_GLOBAL_MEMORY_BUS_WIDTH: 5120
CU_DEVICE_ATTRIBUTE_MEMORY_CLOCK_RATE: 1215
Max Memory bandwidth: 1555200000000 bytes (1555.2 GiB)
CU_DEVICE_ATTRIBUTE_ECC_SUPPORT: true
```

Worker 0:1[1004]: Message: DCGM CudaContext Init completed successfully.

```
CU_DEVICE_ATTRIBUTE_MAX_THREADS_PER_MULTIPROCESSOR: 2048
CUDA_VISIBLE_DEVICES: MIG-GPU-5fd15f35-e148-2992-4ecb-9825e534f253/2/0
CU_DEVICE_ATTRIBUTE_MULTIPROCESSOR_COUNT: 42
CU_DEVICE_ATTRIBUTE_MAX_SHARED_MEMORY_PER_MULTIPROCESSOR: 167936
CU_DEVICE_ATTRIBUTE_COMPUTE_CAPABILITY_MAJOR: 8
CU_DEVICE_ATTRIBUTE_COMPUTE_CAPABILITY_MINOR: 0
CU_DEVICE_ATTRIBUTE_GLOBAL_MEMORY_BUS_WIDTH: 5120
CU_DEVICE_ATTRIBUTE_MEMORY_CLOCK_RATE: 1215
Max Memory bandwidth: 1555200000000 bytes (1555.2 GiB)
CU_DEVICE_ATTRIBUTE_ECC_SUPPORT: true
```

Skipping UnwatchFields() since DCGM validation is disabled

Step 5: Stream metrics via `dcgmi dmon`

Now in another window, we can observe the metrics being attributed to each MIG device using `dcgmi dmon`. Notice that in this example, we show the GPU Activity (1001) and the Tensor Core Utilization (1004) for the group 8 that we created in Step 3.

```
$ dcgmi dmon -e 1001,1004 -g 8
```

#	Entity	GRACT	TENSO
	Id		
	GPU 0	0.000	0.000
	GPU-I 0	0.000	0.000
	GPU-I 1	0.000	0.000
	GPU 0	0.000	0.000
	GPU-I 0	0.000	0.000
	GPU-I 1	0.000	0.000
	GPU 0	0.457	0.442
	GPU-I 0	0.534	0.516
	GPU-I 1	0.533	0.515
	GPU 0	0.845	0.816
	GPU-I 0	0.986	0.953
	GPU-I 1	0.985	0.952
	GPU 0	0.846	0.817
	GPU-I 0	0.987	0.953
	GPU-I 1	0.986	0.953
	GPU 0	0.846	0.817
	GPU-I 0	0.987	0.954
	GPU-I 1	0.986	0.953
	GPU 0	0.843	0.815
	GPU-I 0	0.985	0.951
	GPU-I 1	0.983	0.950
	GPU 0	0.845	0.817
	GPU-I 0	0.987	0.953
	GPU-I 1	0.985	0.952
	GPU 0	0.844	0.816
	GPU-I 0	0.985	0.952
	GPU-I 1	0.984	0.951
	GPU 0	0.845	0.816
	GPU-I 0	0.986	0.952
	GPU-I 1	0.985	0.952

Understanding Metrics

To understand the metrics attribution, GRACT is calculated based on the number of allocated compute resources to the total number of compute resources available on the GPU. We can see the total number of compute resources (SMs) using the `nvidia-smi` command:

```
$ sudo nvidia-smi mig -lgip
```

GPU instance profiles:									
GPU	Name	ID	Instances	Memory	P2P	SM	DEC	ENC	
			Free/Total	GiB		CE	JPEG	OFA	
0	MIG 1g.5gb	19	0/7	4.75	No	14	0	0	
						1	0	0	
0	MIG 2g.10gb	14	0/3	9.75	No	28	1	0	
						2	0	0	
0	MIG 3g.20gb	9	0/2	19.62	No	42	2	0	



[Privacy Policy](#) | [Your Privacy Choices](#) | [Terms of Service](#) | [Accessibility](#) | [Corporate Policies](#) | [Product Security](#) | [Contact](#)

Copyright © 2018-2026, NVIDIA Corporation.

Last updated on 2026-09-10.